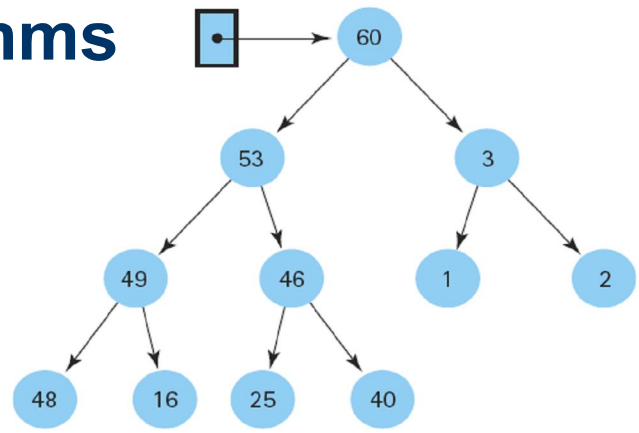
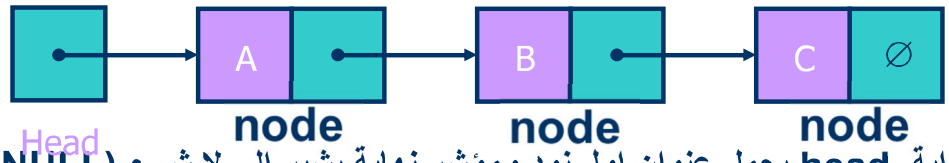
Data Structures & Algorithms



Introduction

: linked list

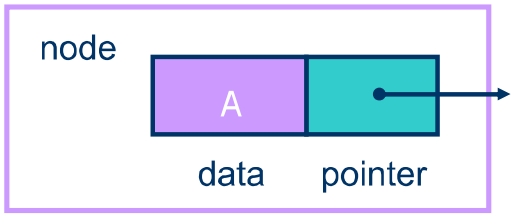
- هي **dynamical data structure** تركيبة بيانات ديناميكية (يمكن تغيير حجمها خلال عملية **compiling**)
- مكونة من مجموعة من **nodes** متصلة ببعضها البعض بواسطة روابط **linkers**



- لها مؤشر بداية **head** يحمل عنوان اول نود ومؤشر نهاية يشير الى لا شيء **(NULL)** .
- **node** هي عبارة عن سجل بيانات يحتوي على حقلين :

١. **data filed** لحفظ البيانات من أي نوع

٢. **linker field** لحفظ عنوان الـ **node** التالية



Linked list properties:

- ليس من الضرورة تحديد الحجم يمكن ادخال البيانات دون تحديد الحجم
- تقبل انواعا غير متجانسة من البيانات بعكس المصفوفات
- غير متجاورة ومتلاصقة بالذاكرة مما يسهل عمليات الحذف والاضافة للعناصر بعكس المصفوفات

Linked list type:

- Single linked list

القائمة المتصلة احادية الاتجاه .

- Double linked list

القائمة المتصلة ثنائية الاتجاه

- Circular linked list

القائمة المتصلة الدائرية

Single linked list

- مؤشر البداية يحمل عنوان اول بينما مؤشر النهاية يشير الى لا شيء

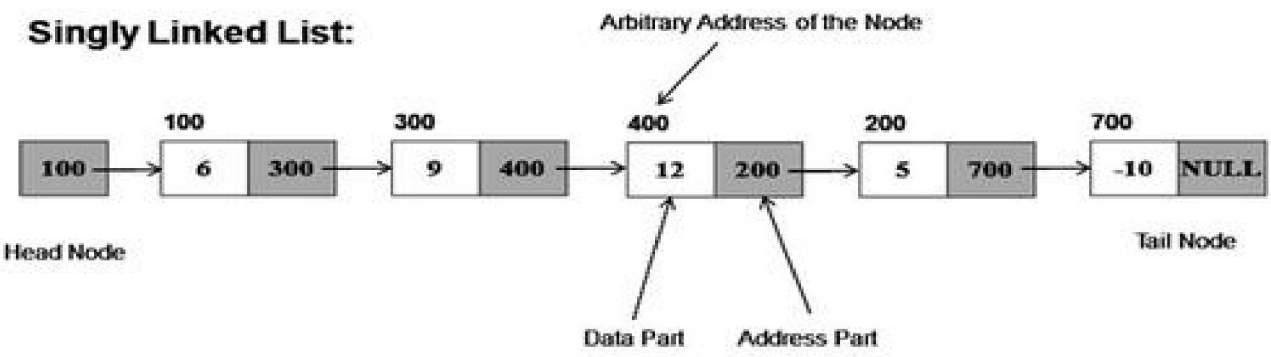


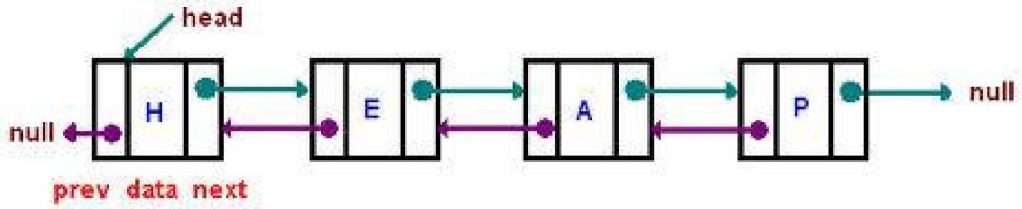
Fig: Singular Linked List

Double linked list

تحتوي على مؤشرين احدهما للنود التالية والاخر للنود السابقة

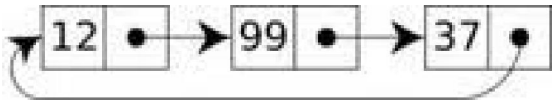
النود الاولى احد مؤشريها لها يشير الى لاشيء والثاني للنود التالية

النود الاخيرة احد مؤشريها لها يشير الى لاشيء والثاني للنود السابقة



Circular linked list

- يكون موشر النود الاخيرة تشير الى اول نود



linked list note:

- لتمثيل نود باستخدام لغة السي نستخدم structure

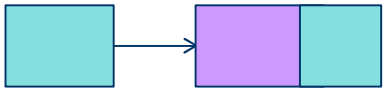
```
struct node{
     datatype data;
     node *next;
};
```

- معاملات التعامل مع الذاكرة ومنها new & delete حيث يستخدم الاول لحجز مساحة في الذاكرة والاخر لتحريره

```
datatype *ptr = new datatype;
delete ptr;
```

- نحجز موقع في الذاكرة لاول نود ولتحقيق ذلك نستخدم معاملات التعامل مع الذاكرة :

```
node *head = new node;
```



- بعد حجز موقع لاول نود يفضل ان نجعل قيمة ماتشير اليه NULL
- للوصول الى خصائص أي نود نستخدم المعامل ->

nodename -> data العنصر الموجود في nodename

nodename -> next النود التالية للـ nodename

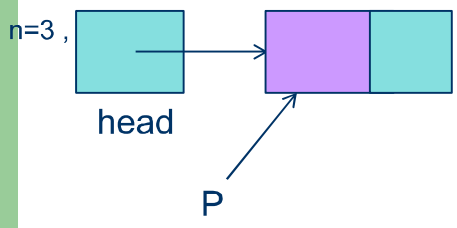
Single linked list operation :

- `CreateList` : input numbers to list
- `CountingList` : how many node in list
- `PrintList`: print all the nodes in the list
- `InsertNode`: insert a new node at a particular position
- `DeleteNode`: delete a node with a given value
- `IsEmpty`: determine whether or not the list is empty
- `FindNode`: find a node with a given value

Create List algorithms: :

الخطوات :

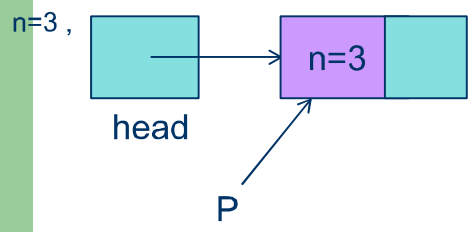
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

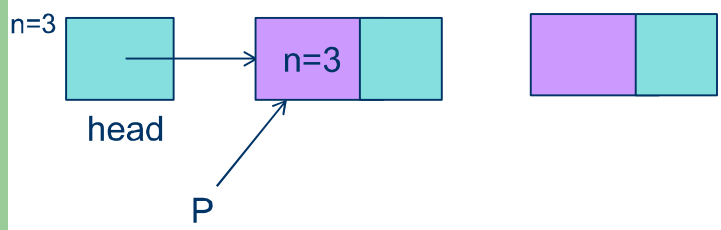
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

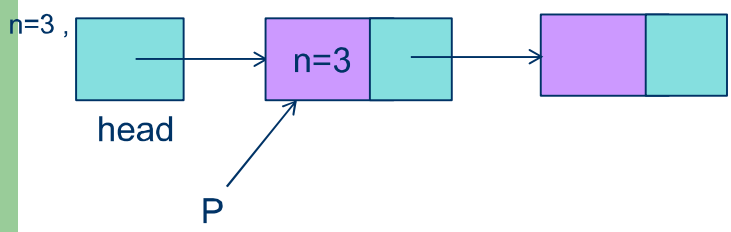
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

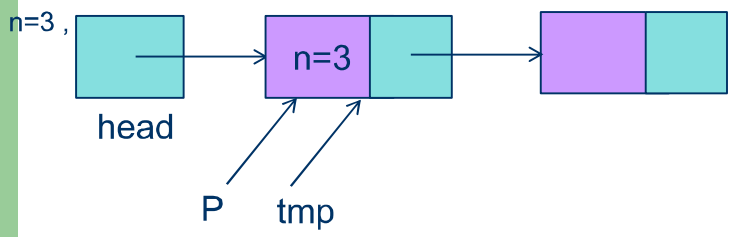
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

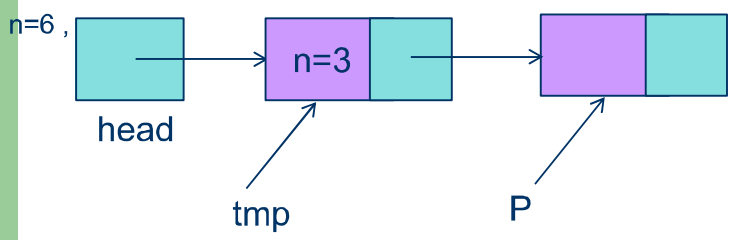
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

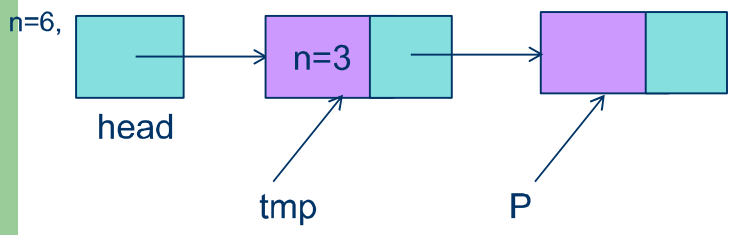
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

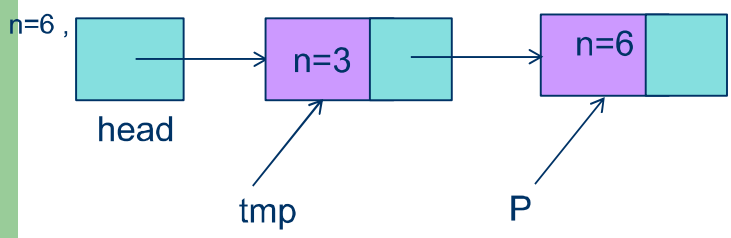
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

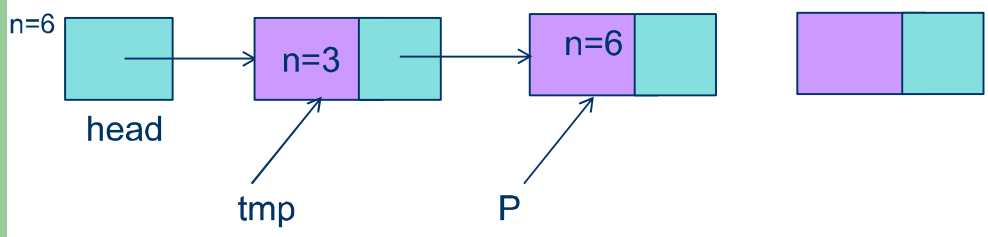
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

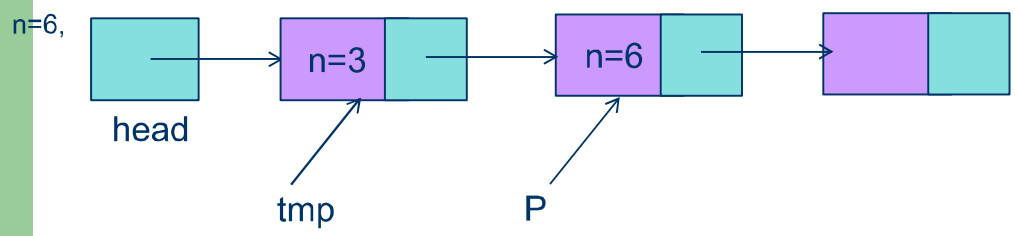
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

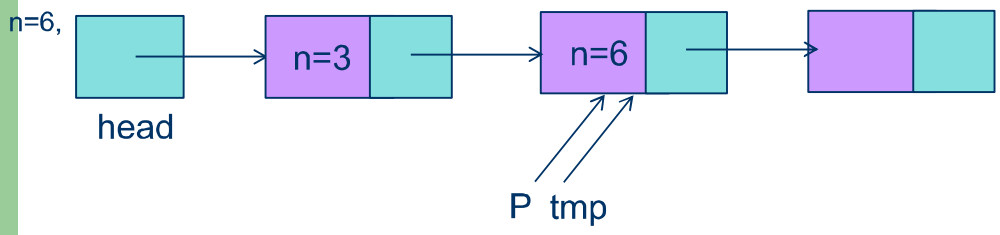
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

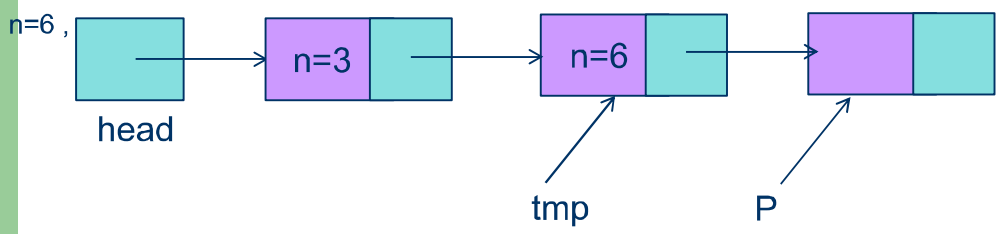
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

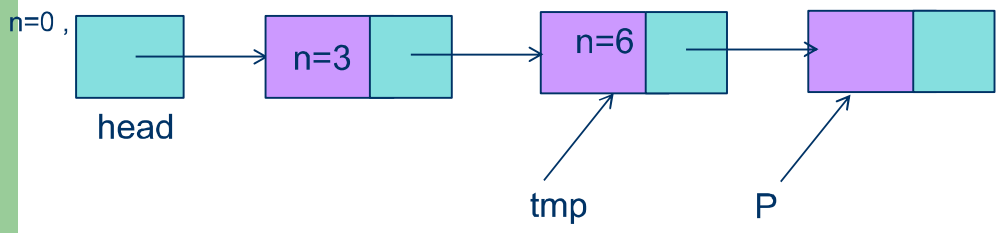
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

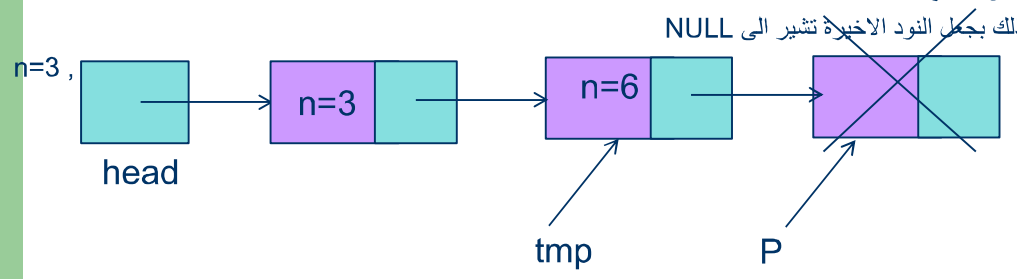
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

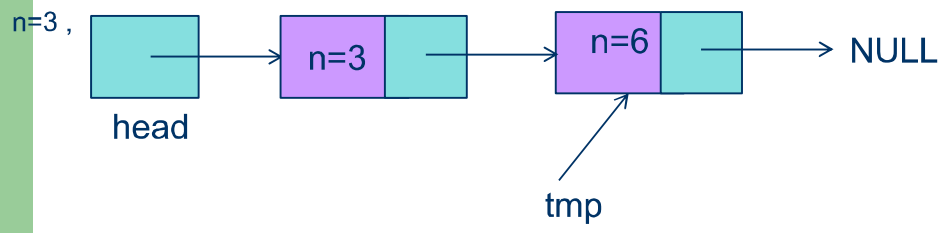
١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



Create List algorithms: :

الخطوات :

١. ندخل القيمة
٢. اذا كانت صفر انتقلنا الى ٨ واذا غير ذلك نذهب الي ٣
٣. نخزن القيمة با لنود الحالية
٤. ننشا نودا جديدة لتخزين العدد التالي
٥. نربط النود الحالية بالنود الجديدة
٦. نتحرك الى النود الجديدة لكي يتم إضافة مدخلاته
٧. نعود الى الخطوة ١
٨. نحرر اخر نود من الذاكرة
٩. ننهي القائمة وذلك بجعل النود الاخيرة تشير الى NULL



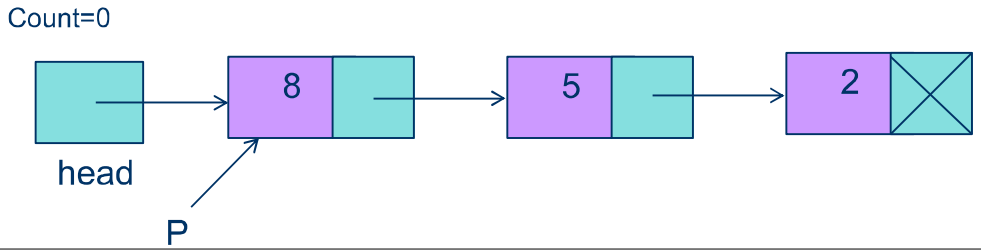
CreateList Code :

```
void CreatList (node *p) {
    int n;
    node *tmp=NULL;
    for(;;)
        {
            cout<<"Enter the value\n";
            cin>>n;
            if(n==0)
                {
                    delete tmp->next;
                    tmp->next = NULL;
                    break;
                    }
            else
                {
                    p->data=n;
                    p->next=new node;
                    tmp = p;
                    p=p->next;
                    }
            }
    }
```

Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان عنوان مايشير له NULL نذهب الى ٧ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

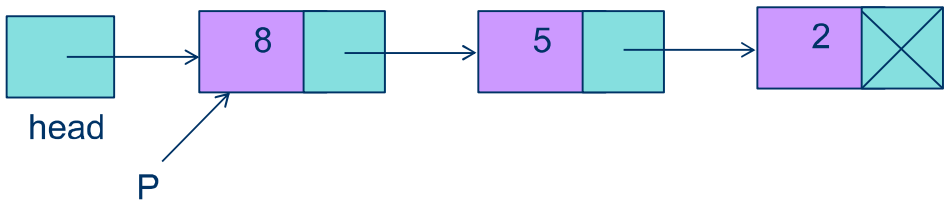


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٧ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=0

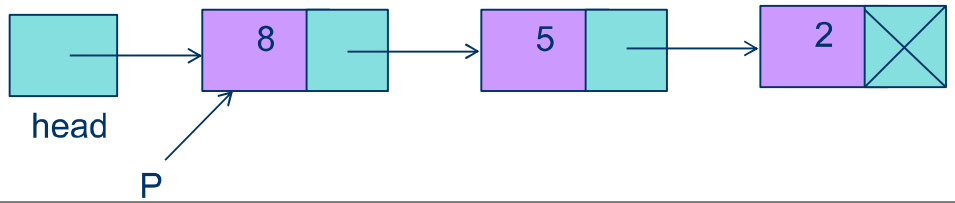


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=1

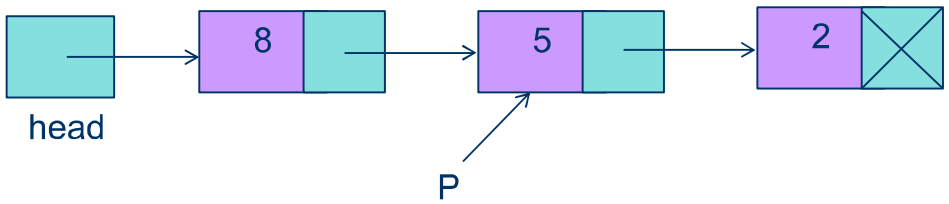


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=1

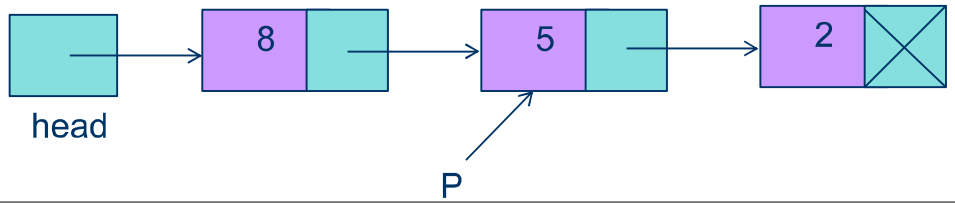


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=1

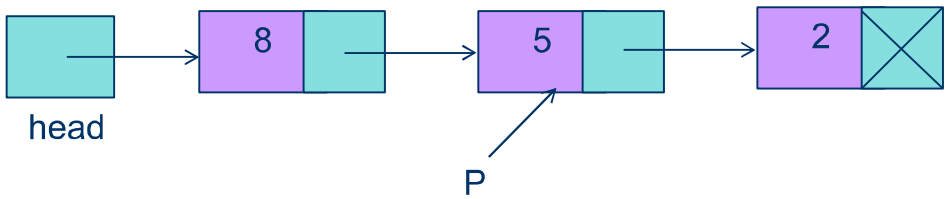


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=2

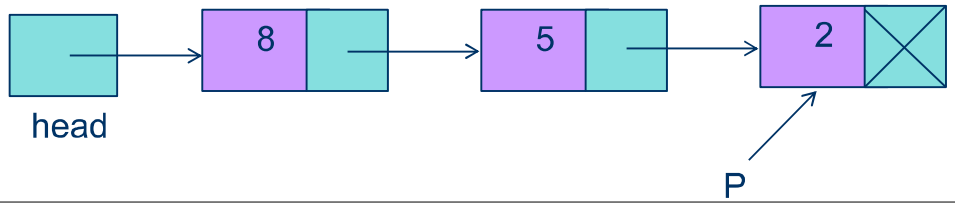


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=2

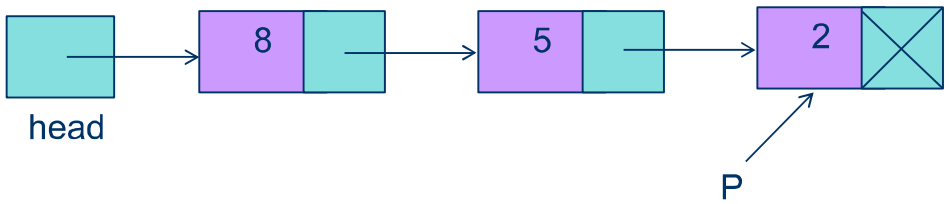


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٧ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

Count=2

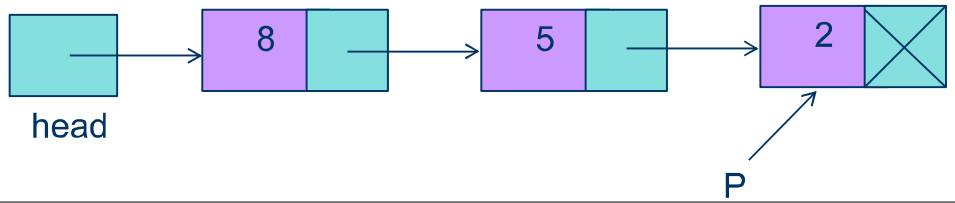


Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف

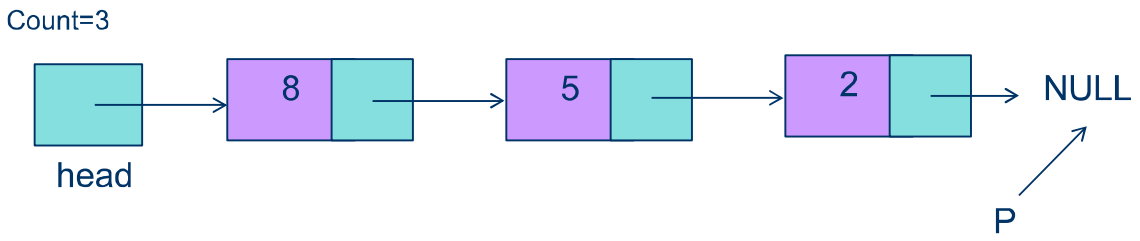
Count=3



Counting List algorithms:

الخطوات :

١. ابدا
٢. نجعل القيمة البدائية للعداد بصفر
٣. نتحقق من مؤشر النود الحالية اذا كان يشير لـ NULL نذهب الى ٦ فيما عدا ذلك نذهب الى ٤
٤. نزيد العداد بواحد
٥. ننتقل للنود التالية
٦. نعود للخطوة ٣
٧. نقوم بارجاع العداد للدالة الرئيسية
٨. نتوقف



Counting List code:

```
int CountList(node *p){
    int count=0;
    while (p!=NULL)
    {
        count++;
        p=p->next;
    }
    return count;
}
```

Print List algorithms :

١. نختبر مؤشر النود الحالية اذا كان عنوان مايشير له NULL نذهب ٥ الى فيما عد ذلك نذهب الى ٢
٢. نطبع قيمة النود الحالية
٣. نتحرك الى النود التالية
٤. نعود للخطوة ١
٥. نتوقف

Print List code:

```
void PrintList(node *p)
{
    while(p!=NULL)
    {
        cout<<p->data<<"\t";
        p=p->next;
    }
    cout<<endl;
}
```

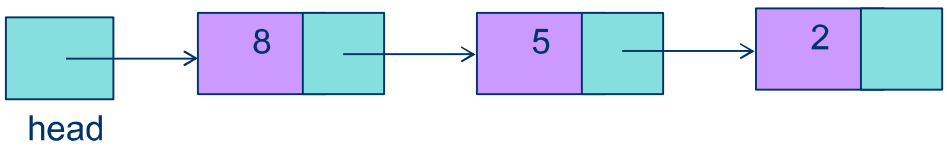
Insert new Node

- Possible cases of `InsertNode`
  1. Insert into an empty list
  2. Insert in front
  3. Insert at back
  4. Insert in middle
- But, in fact, only need to handle two cases
  - Insert as the first node (Case 1 and Case 2)
  - Insert in the middle or at the end of the list (Case 3 and Case 4)

Insert as the first node algorithm :

الخطوات :

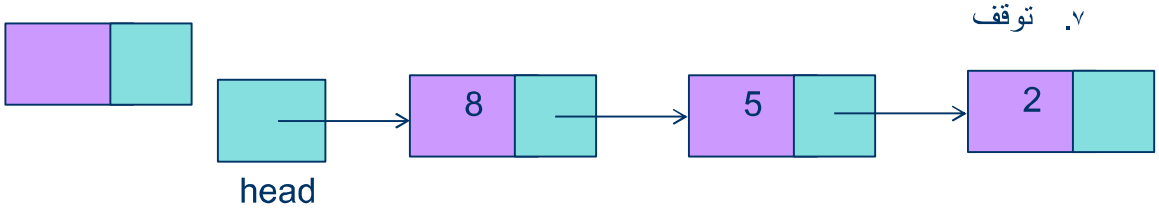
١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف



Insert as the first node algorithm :

الخطوات :

١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف

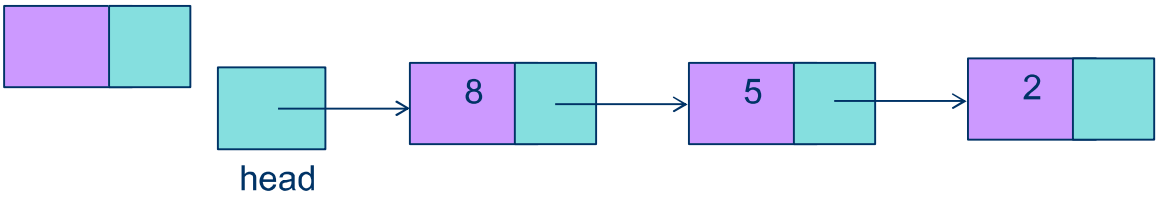


Insert as the first node algorithm :

الخطوات :

١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف

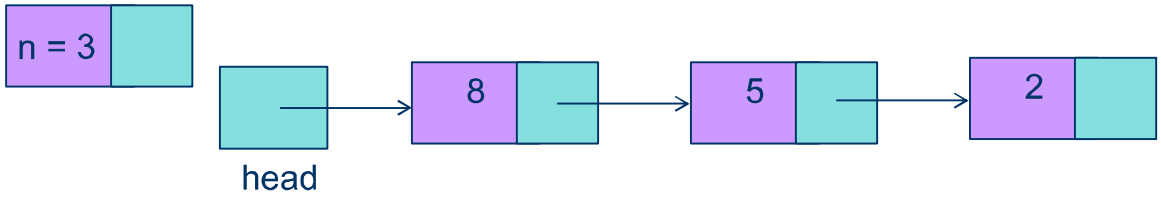
n = 3



Insert as the first node algorithm :

الخطوات :

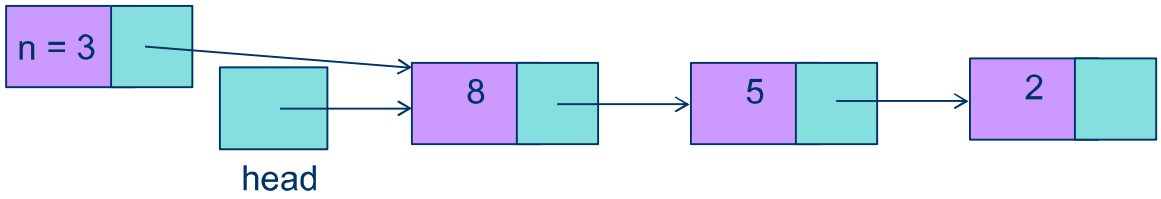
١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف



Insert as the first node algorithm :

الخطوات :

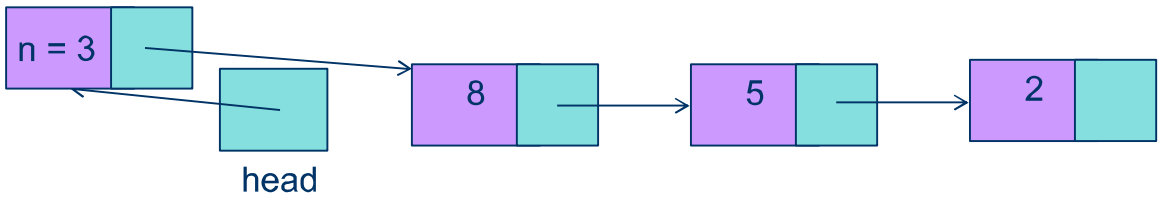
١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف



Insert as the first node algorithm :

الخطوات :

١. ابدا
٢. ننشا نود جديدة باستخدام المعامل new
٣. نقرا القيمة n
٤. ندخل n في حقل بيانات النود الجديدة
٥. نسند في حقل عنوان النود الجديدة عنوان النود الاولى
٦. نعيد النود الجديدة لتكون اول نود في القائمة
٧. توقف



Insert as the first node code :

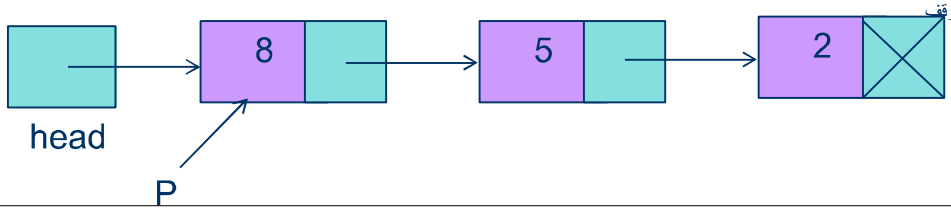
```
node *isertatbeganing(node *p){
    int num;
    cout<<"Enter the value you want to insert at beganing:\n";
    cin>>num;
    node *newnode=new node;
    newnode->data=num;
    newnode->next=p;
    return newnode;
}
```

Insert in the middle or at the end of the list (any at list) algorithms :

- الخطوات

١. ابدا
٢. ندخل القيمة المراد اضافة نود بعدها loc
٣. نعرف مؤشر locadd للنود المطلوب الاضافة بعدها ونسند القيمة الابتدائية له NULL
٤. نتحقق من عنوان المؤشر للنود الحالية اذا مساوية للـ NULL نذهب الى ٩
٥. نتحقق من قيمة النود الحالية اذا كانت مساوية للـ loc نذهب الى ٨
٦. ننتقل الى النود التالية
٧. نعود الى ٤
٨. نسند للمؤشر locadd عنوان النود الحالية
٩. نختبر قيمة locadd اذا كانت مساوية للـ NULL نذهب الى ١٤
١٠. نقرا n
١١. ننشا نود جديدة باستخدام المعامل new
١٢. نسند n الى حقل بيانات النود الجديدة
١٣. نربط النود الجديدة بالنود السابقة واللاحقة
١٤. نذهب الى الخطوة ١٥
١٥. نطبع ("not found")
١٦. نتوقف

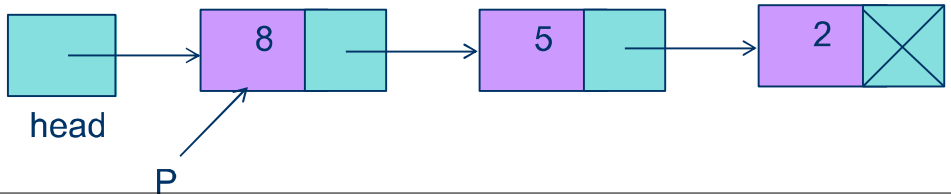
loc = 5



Insert in the middle or at the end of the list (any at list) algorithms :

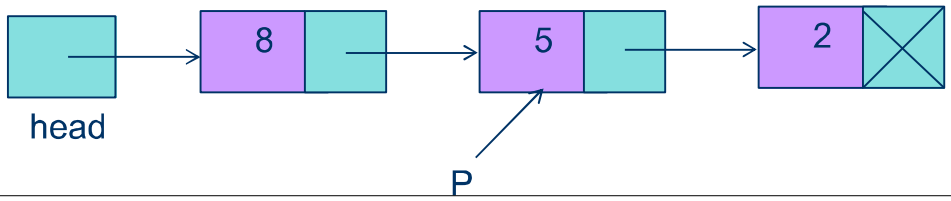
loc = 5

5 != 8



Insert in the middle or at the end of the list (any at list) algorithms :

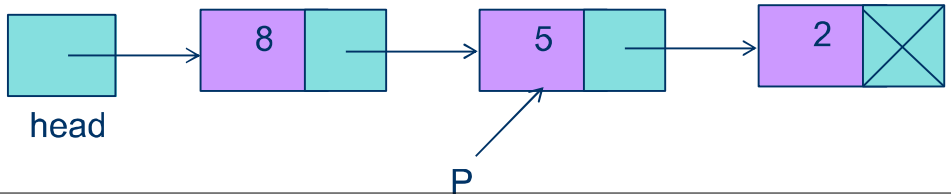
loc = 5



Insert in the middle or at the end of the list (any at list) algorithms :

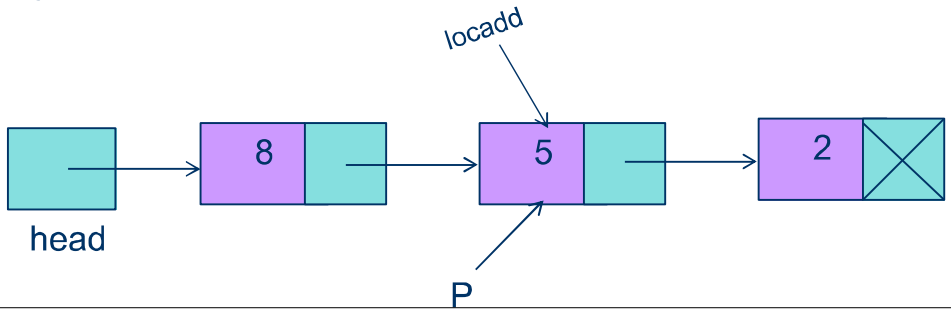
loc = 5

5 == 5



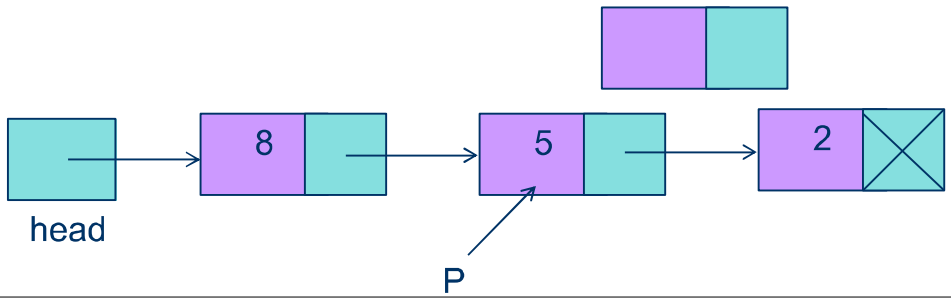
Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5



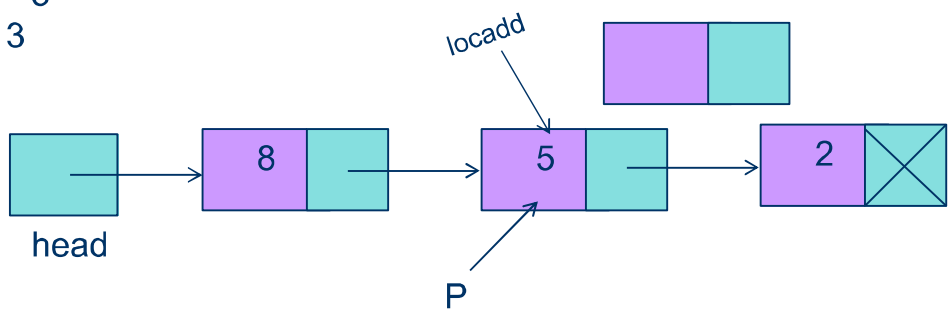
Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5



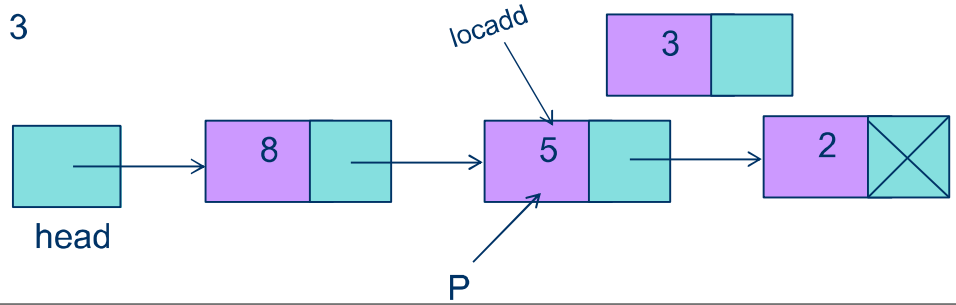
Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5
n = 3



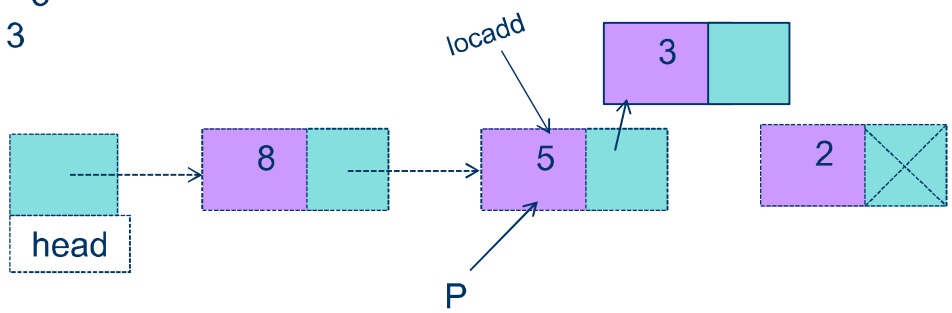
Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5
n = 3



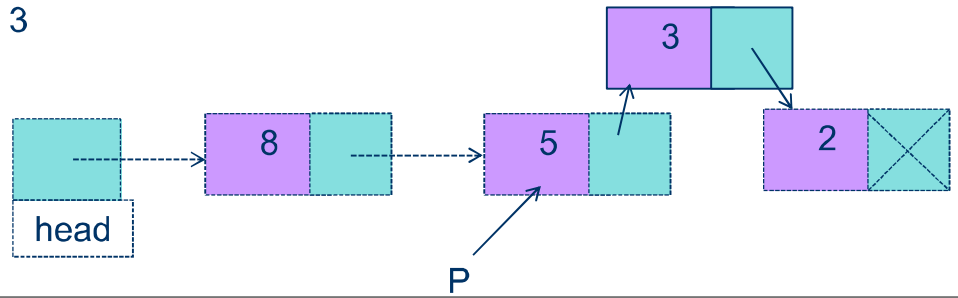
Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5
n = 3



Insert in the middle or at the end of the list (any at list) algorithms :

loc = 5
n = 3



Insert in the middle or at the end of the list (any at list) code:

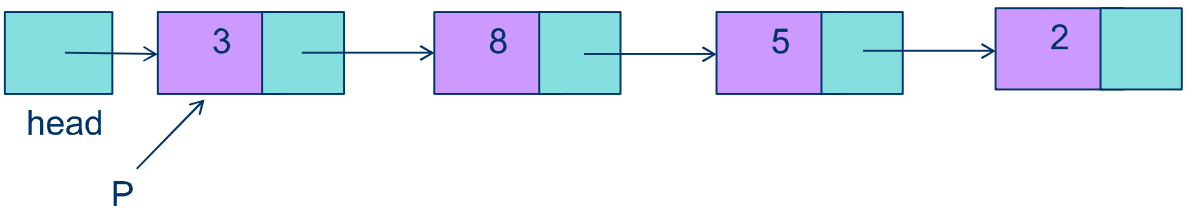
```
void inseratloc(node *p,int loc){
    int num;    node *locadd,*temp; locadd=NULL;
    while(p!=NULL)
    {
        if(p->data==loc)
        {
            locadd=p;           break;
        }
        else
            p=p->next;
    }
    if(locadd==NULL)
        cout<<"not found\n";
    else
    {
    cout<<"Enter value to isert it:\n"; cin>>num;

    node *newnode=new node; newnode->data=num;

    temp=p->next;   p->next=newnode;    newnode->next=temp;
    }
}
```

Delete Node algorithm:

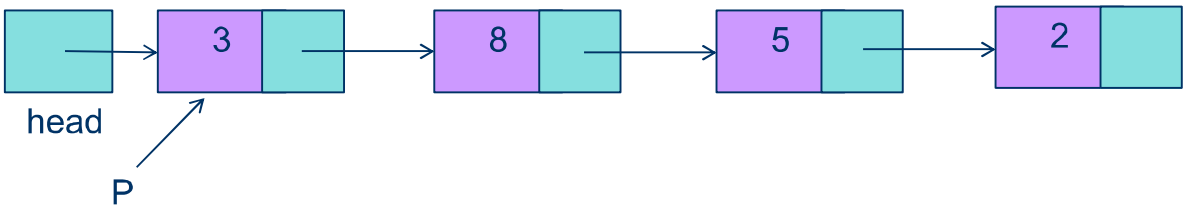
Key = 5



Delete Node algorithm:

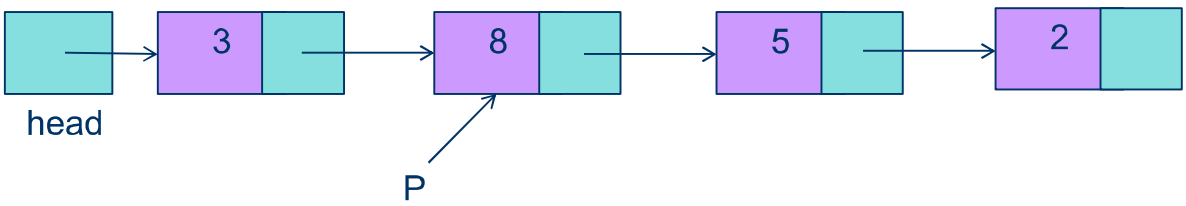
Key = 5

3 != 5



Delete Node algorithm:

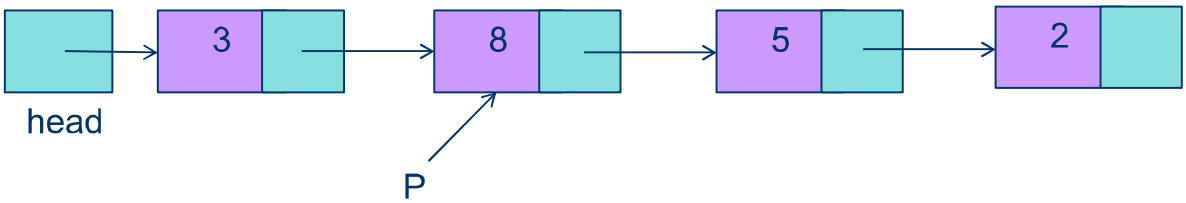
Key = 5



Delete Node algorithm:

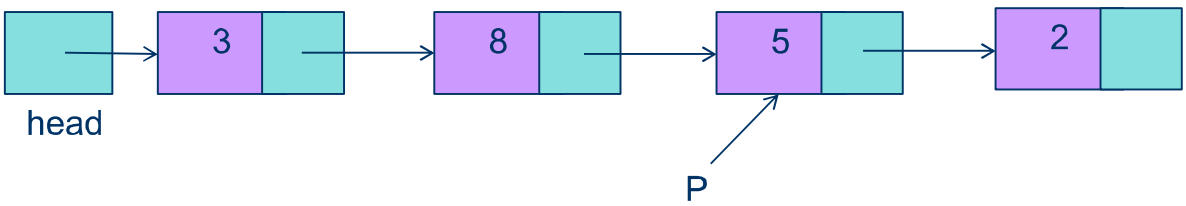
Key = 5

8 != 5



Delete Node algorithm:

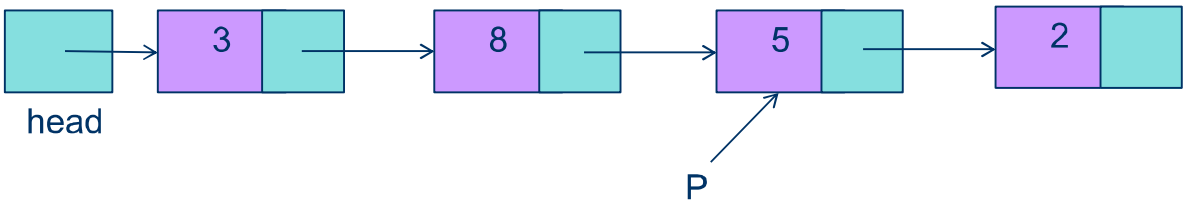
Key = 5



Delete Node algorithm:

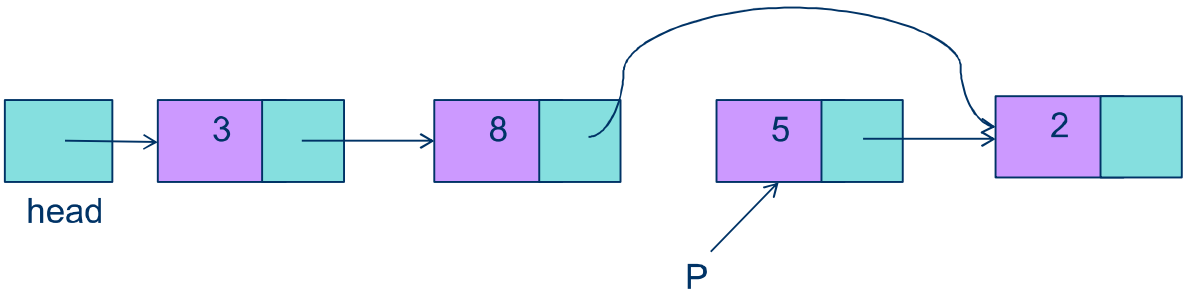
Key = 5

5 == 5



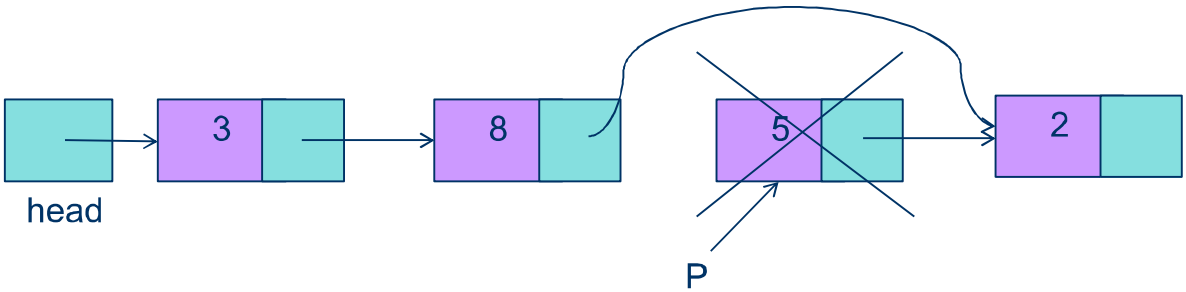
Delete Node algorithm:

Key = 5



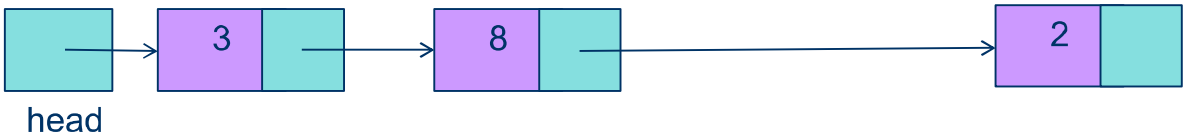
Delete Node algorithm:

Key = 5



Delete Node algorithm:

Key = 5



Delete Node algorithm:

الخطوات :

١. نقرا قيمة المراد حذفه key
٢. نتحقق من النود الحالي هل تساوي قيمته الـ key اذا نعم نذهب الى ٣ واذا لا نذهب الى ٧
٣. نتحقق من النود اذا كانت النود الاولى نذهب الى ٤ واذا لا ٥
٤. نعيد النود التالية للدالة الرئيسية
٥. تربط النود السابقة للنود المطلوب حذفها مع النود اللاحقة للنود المطلوب حذفها
٦. نحذف النود ثم نذهب الى الخطوة ٩
٧. نتحرك الى النود التالية
٨. نعود الى الخطوة ٢
٩. نعيد نود البداية

Delete Node Code:

```
node *deletenode(node *p,int key){
    node *start,*temp;  start=p;
    while(p!=NULL)
    {
        if(p->data==key)
            if(start==p)
                return start->next;
            else
            {
                temp->next=p->next;     delete p;       return start;
            }

        else
            {
                temp=p;     p=p->next;
            }
        }
    cout<<"not found "<<key<<endl;
    return start;
}
```

Main Function:

```
void main (){
    int loc,key,choice,n;  char answer;     node *head=new node;

    cout<<"1. to creat list\n"     <<"2. to print list\n"         <<"3. to counting list\n"
        <<"4. to add at beganing\n"<<"5. to add at any location\n"<<"6. to delet any node\n";
    cout<<"want to try (y/n)\n";    cin>>answer;

    while(answer=='Y'||answer=='y') {//start

        cout<<"Enter your choise:\n"; cin>>choice;

    switch(choice)  { // start switch
    case 1:CreatList(head);  break;
    case 2:cout<<"linked lists nodes are:\n";  PrintList(head); break;
    case 3:n=CountList(head);       cout<<"number of node "<<n<<endl;break;
    case 4:head=isertatbeganing(head);      break;
    case 5:cout<<"Enter value you want to inser after it:\n";cin>>loc;inseratloc(head,loc);break
    case 6:cout<<"Enter node value to delete it:\n"; cin>>key; head=deletenode(head,key);break;
    default:cout<<"invalide choise\n";}//end switch

    cout<<"more ?(y/n)\n";  cin>>answer;    }/*end while */
}
```

Question :

- How to reverse linked list elements?

END